One fixed-structure equation system for the CPU and the GPU

Bringing together OLF's alternative equations, the GPU superset matrix and the upstream scatter approach

olf-gpu design note

November 2026

Abstract

Two pieces of work solve the same problem independently. On the CPU, the *alternative equations* branch of the OLF fork keeps the Jacobian sparsity pattern constant across PV/PQ switches and contingencies, so that the symbolic factorisation is built once. On the GPU, this repository builds a *superset* matrix because a uniform batch requires one pattern for every contingency. A third piece, upstream draft PR #1521 ("Equation array, scatter approach"), replaces the term-by-term walk of the equation arrays by precomputed index maps, which is what the GPU kernels already use.

This note describes the three, shows where they agree and where they differ, and proposes a plan to make OLF own a single fixed-structure description that its own solver runs on and that the GPU reads as plain arrays instead of re-deriving it. Nothing in this note has been measured yet; phase 0 of the plan is the measurement.

Contents

1 The problem both sides solve **2**

2 Approach A: alternative equations on the CPU **2**

3 Approach B: the fixed superset matrix on the GPU **4**

4 Upstream PR #1521: the scatter approach **5**

5 Side by side **6**

6 Target design **6**

6.1 Slots owned by variables 6

6.2 The read-only contract 7

6.3 What stays GPU-specific 8

7 Plan **8**

8 Risks and open questions **10**

1 The problem both sides solve

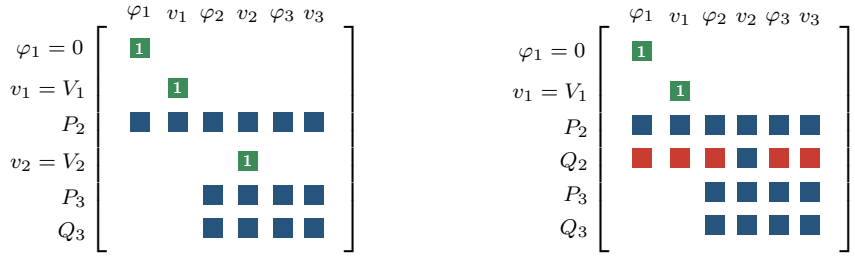
OLF's AC equation system has one *column* per active equation and one *row* per variable. It stores J^T in compressed-column form; read row by row, the same arrays are the true Jacobian J in compressed-row form. The figures below show J: one row per equation, one column per variable.

The set of active equations is not constant:

- a generator reaching a reactive limit replaces its voltage equation by a reactive balance (PV→PQ);
- a contingency removes the equations and variables of the buses it islands, and the terms of the branches it opens;
- a remedial action or a control outer loop adds or removes control equations (tap, shunt section, phase shift) together with their variables.

Each of these changes the sparsity pattern, the column numbering, or both. On the CPU the cost is a new symbolic factorisation. On the GPU it is worse: a uniform batch factorises many matrices in one call and requires them to share one pattern, so a pattern that depends on the contingency cannot be batched at all.

Running example. Three buses in a chain, $1-2-3$. Bus 1 is the slack, bus 2 holds a generator in voltage control, bus 3 is a load. The variables are $(\varphi_1, v_1, \varphi_2, v_2, \varphi_3, v_3)$.



(a) bus 2 in voltage control

(b) after PV→PQ at bus 2

Figure 1: Legacy modelling. The switch replaces the equation $v_2 = V_2$ by Q_2 and adds five nonzeros (red). The pattern changed: the symbolic factorisation is rebuilt.

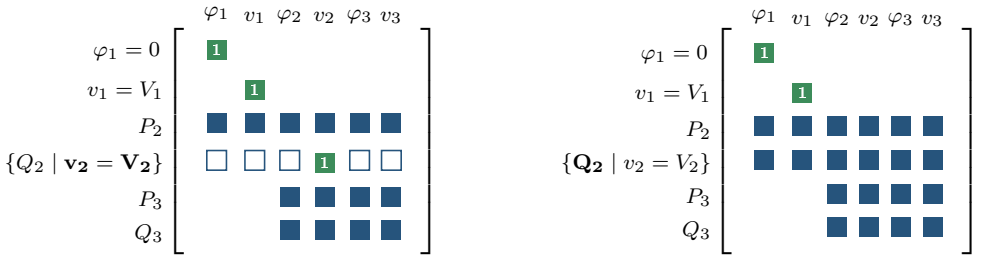
2 Approach A: alternative equations on the CPU

Where. OLF fork, branch `complementary-equations` (one squashed commit, July 2026) and its continuation `experiment/alt-eq-contingency-fallback`. Opt-in through the `alternativeEquations` parameter of `OpenLoadFlowParameters`. Not part of `olf-local-patches`, and 49 commits behind upstream main at the time of writing.

Idea. An equation owns one matrix column but carries several *alternative bodies*; exactly one is active. The pattern contributed by the equation is the union of the patterns of all its alternatives. Entries of the inactive alternatives are stored as explicit zeros. Switching the active alternative is a value update.

What is implemented.

- `AlternativeEquation` (scalar system) and per-element alternatives inside `EquationArray` (vectorised system), including an `elementActiveAlternativeNum[]` array.

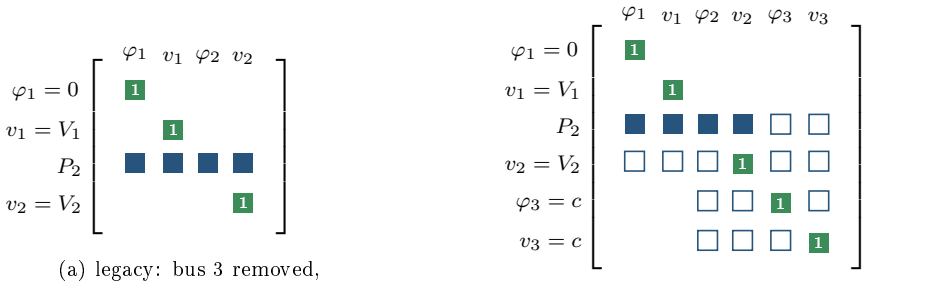


(a) voltage alternative active

(b) reactive alternative active

Figure 2: Alternative equations. Hollow squares are stored zeros. Both states share one pattern, so (a)→(b) changes values only.

- On the reactive equation of a controller bus: reactive balance, voltage target of the controlled bus (local or remote), and reactive distribution (`DISTR_Q`) when several controllers share the control. The first enabled controller carries the voltage target.
- A *disabled* alternative on P and Q of every eligible bus ($\varphi = \text{const}$, $v = \text{const}$), used when a contingency islands the bus (figure 3).
- Terms of opened branches stay registered: their entries become stored zeros.
- A *partial Jacobian update*: after a contingency, restore a snapshot of the values and re-derive only the touched columns. The branch's commit notes credit it with about a fifth of a large Pegase security analysis.



(a) legacy: bus 3 removed, 4×4 system

(b) fixed structure: same 6×6 pattern, rows of bus 3 hold identities

Figure 3: Outage of line $2 - 3$, which islands bus 3.

Where it stops. The following stay on the legacy modelling, either by eligibility rule or by falling back:

- buses touching a zero-impedance branch, a converter, or flagged fictitious;
- buses involved in transformer or shunt voltage control, and the whole run when either control is enabled in the parameters;
- merged or hidden voltage controls, generators with slope, reactive power control;
- the slack: its angle equation is still a separate equation and its P equation is deactivated, so a slack relocation is structural;
- any solver other than `NEWTON_RAPHSON`, asymmetrical runs, secondary voltage control.

When a contingency reconfigures a control the alternatives cannot follow, the system is no longer square. The branch detects this (`EquationSystemNotSquareException`) and re-runs that contin-

gency on a legacy context. Contingencies are reordered so that the structure-preserving ones run first and share one construction.

3 Approach B: the fixed superset matrix on the GPU

Where. This repository. Rationale in `docs/gpu-vs-olf.md` §1–2; construction in `native/cuda/structure/shared_structure.cu`; host inputs in the `ac/equations/vector/gpu` package and its `fixedmatrix` sub-package.

Idea. The device builds, once, from the base network, the union of everything any contingency or outer-loop decision may activate. A contingency then changes only values, per lane of the batch.

Stable rows. A row belongs to a *variable*, not to an equation: the P row of a bus is the row of its φ variable, its Q row the row of its v variable. Control variables (tap ratio, phase shift, shunt susceptance, the dummy variables of a coupler) own a row too. The system is square by construction.

Row mode. A per-row, per-lane code says which equation the row currently holds (`GpuRowMode`): `FIXED_VARIABLE` (x = target), `EQUATION`, `BUS_TARGET_V_WITH_SLOPE`, `BRANCH_TARGET_P`, `BUS_TARGET_Q`, `CROSS_FIXED_VARIABLE`.

Element mask. A per-lane mask disables outaged branches, shunts and HVDC links; their entries stay in the pattern.

Relocation. Some equations are written on another variable's row: the voltage equation of a transformer-controlled bus sits on the tap row; a remote voltage control is carried by one controller's v row, chosen per lane among the candidates; a coupler-tree child's balance sits on the coupler's dummy rows. The pattern holds the union of all candidate placements.

Role-folded destinations. For every term output the host precomputes a destination: a Jacobian value slot, a mismatch row, or "skip". The fill kernels only read these arrays.

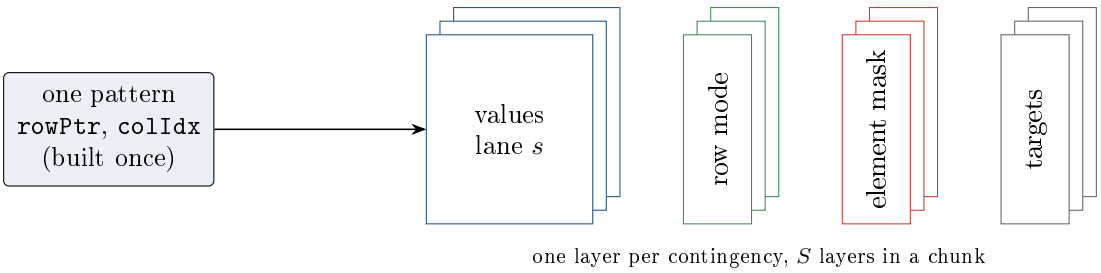


Figure 4: The batched security analysis. The pattern is shared; a contingency is a set of per-lane arrays.

What it costs today. The host does not read this structure from OLF: it re-derives it. The extractor validates OLF's equation system, then rebuilds row placement, row modes, targets and voltage-control roles from the `LfNetwork`, and the contingency planners mirror OLF's contingency semantics a second time. Rough sizes:

Layer	Lines
Java extraction, activation, relocation (`vector/gpu`, `fixedmatrix`)	∼3 100
Native pattern construction (`structure/`, `fixedmatrix/`)	∼3 400
Java contingency planning that mirrors OLF (`batch/plan`)	∼4 700
Whole repository: Java `src/main` / native `cuda`	31 900 / 27 200

The code comments record the consequence: several bugs came from the mirror disagreeing with OLF, one of which put 4 183 of 7 494 contingencies wrong.

4 Upstream PR #1521: the scatter approach

What it is. A draft PR by Geoffroy Jamgotchian against upstream main, one commit, touching `EquationArray`, `EquationDerivativeVector` and `EquationTermArray` (plus a benchmark). Its description is empty; what follows is read from the diff.

What it does. Before, evaluating an equation array *gathered*: for each equation, walk its terms and fetch each term's value, jumping across the term vectors. The PR *scatters* instead: read each term vector sequentially and write each value to a precomputed destination.

- `eval`: per term array, two parallel arrays `evalScatterTermElementNums` and `evalScatterColumns`. Inactive terms and terms of inactive equations are filtered out when the arrays are built, so the loop is branch-free.
- `der`: one `Scatter` per term derivative vector, mapping term element → position in the flattened derivative vector. Inactive terms write zero.
- The index maps are invalidated whenever activity or numbering changes.

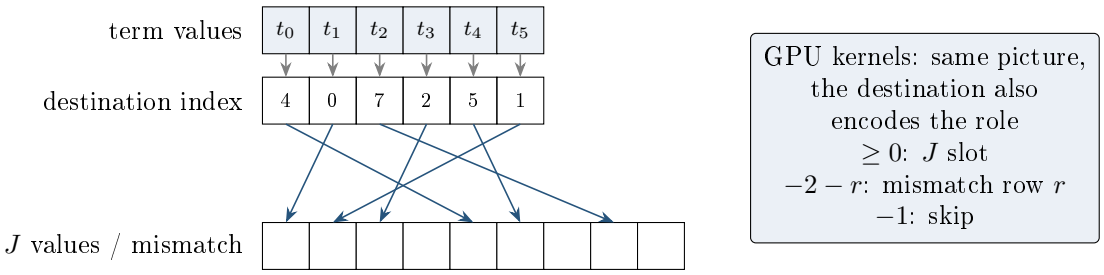


Figure 5: Scatter. The CPU PR and the GPU fill kernels use the same data structure: a destination index per term output.

Why it matters here. It is the third leg of the same design.

- The PR's scatter arrays and the GPU's role-folded destination arrays describe the same mapping. If OLF owns the map, the GPU no longer has to rebuild it from the generated scatter tables and its own row placement.
- The PR's weak point is invalidation: every activation change rebuilds the `eval` maps. Under a fixed structure nothing is ever filtered out, so the maps are built once per network and never invalidated. Alternative equations make the scatter approach cheaper, and the scatter approach gives alternative equations their natural storage.
- The PR's `der` scatter stops at the flattened derivative vector; the last hop to the matrix value array still goes through a handler call per entry. Composing the two gives a direct "term derivative → Jacobian value index" array, which is exactly what a device needs.

5 Side by side

	CPU alternative equations	GPU superset matrix
Fixed pattern	union of all alternatives	union of every placement any lane may use
Slot identity	a *primary equation* owns a column	a *variable* owns a row
Current content of a slot	active alternative	row mode
PV→PQ	switch to the reactive alternative	`FIXED_VARIABLE` → `BUS_TARGET_Q`
Islanded bus	disabled alternative	`FIXED_VARIABLE` in that lane
Remote voltage control	first enabled controller carries the target, others `DISTR_Q`	carrier row chosen per lane, `DISTR_Q` pack
Branch outage	term inactive, variables kept	element mask
Slack	legacy (separate equation)	`FIXED_VARIABLE` on the φ row, relocatable
Transformer / shunt voltage control	excluded; run falls back	relocation to the tap / susceptance row
Zero-impedance branches	excluded	dummy rows, per-lane spanning tree
State of a contingency	events mutating one live system	plain arrays per lane
When it cannot cope	re-run on a legacy context	send the contingency to the CPU
Fill	term walk (scatter with #1521)	scatter through destination arrays

The three real differences.

1. **Slot identity.** Attaching a column to an equation is what makes the CPU branch fragile: control equations and control variables appear and disappear together, and a missed case leaves the system non-square. Attaching the slot to the variable makes squareness structural. Every GPU "relocation" is then simply an alternative of the slot of the variable whose row hosts it.
2. **Coverage.** The GPU already handles, inside the fixed structure, every case the CPU branch excludes. `docs/gpu-vs-olf.md` §2 is, in effect, the specification of a complete alternative-equations model.
3. **Mutation versus data.** The CPU applies a contingency to the `LfNetwork` and lets events update the equation system. The GPU needs the contingency as data, for many contingencies at once. A shared description of a contingency must therefore be computable without applying it.

6 Target design

6.1 Slots owned by variables

Every variable to solve owns exactly one slot. A slot has an ordered list of alternatives and one active alternative. Proposed alternatives per variable type:

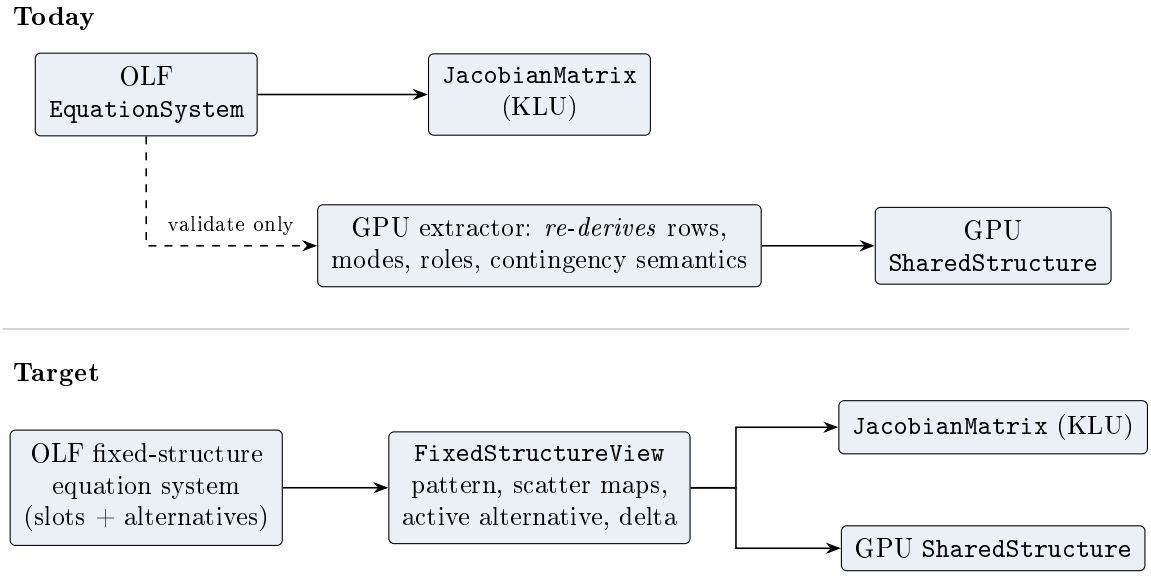


Figure 6: Today the GPU rebuilds what OLF knows. In the target, both solvers read one description.

Variable	Alternatives of its slot
`BUS_PHI`	active balance P; $\varphi = \text{const}$ (slack, reference, disabled or islanded bus); slack distribution among several slacks
`BUS_V`	reactive balance Q; local voltage target; voltage target with slope; voltage target of a remote controlled bus; `DISTR_Q`; $v = \text{const}$ (disabled bus)
`BRANCH_RHO1`	tap fixed; voltage target of the controlled bus; `DISTR_RHO`; reactive balance of the controlled bus
`SHUNT_B`	susceptance fixed; voltage target of the controlled bus; `DISTR_SHUNT_B`
`BRANCH_ALPHA1`	phase shift fixed; `BRANCH_TARGET_P` of the controlled branch
`DUMMY_P`, `DUMMY_Q`	dummy fixed (coupler open or off the tree); `ZERO_PHI` / `ZERO_V`; the relocated balance of a coupler-tree child

The list is the GPU's current row placement rules written as alternatives. Whether every relocation the GPU performs can be expressed this way is the main thing phase 2 must prove; the cross identity (`CROSS_FIXED_VARIABLE`) and the zero-diagonal repair are the cases to look at first, since they exist for factorisation without pivoting rather than for modelling.

6.2 The read-only contract

OLF exposes one immutable description per equation system, plus a small mutable state:

Numbering. Slot index = variable row. Stable for the life of the system.

Pattern. `rowPtr[]`, `colIdx[]`: the union pattern. OLF's own sparse matrix uses the same arrays.

Alternatives. Per slot: the list of alternative types, the element each one targets (for the target vector), and the range of its entries in the pattern.

Scatter maps. Per term array and per output (value, each derivative): the destination index, as in #1521 but composed down to the Jacobian value index and the mismatch row, and tagged with the alternative it belongs to.

State. `activeAlternative[slot]` and `elementActive[element]`. These two arrays are the only things a PV/PQ switch or a contingency changes.

Delta. A contingency or action expressed as a list of changes to the state arrays and to the targets, computed from a `PropagatedContingency` without applying it to the network.

The CPU solver fills its matrix from the scatter maps and the state arrays. The GPU uploads the pattern and the maps once, and one state array per lane.

6.3 What stays GPU-specific

- The physics: already single-sourced by `java2cuda`.
- Everything needed to factorise without pivoting: the row permutation that pairs each row with a dominant diagonal, the zero-diagonal repair, the cross identity, adaptive refinement. They become data *derived from* the shared pattern.
- The device outer loops and the batched solver.
- Packing of physical parameters (pi-model values, limits). It could later read the term arrays' own parameter vectors; that is a separate step.

7 Plan

Each phase ends on something measurable, and the first three have value for the CPU alone, which is what makes them proposable upstream. Durations are rough estimates.

Phase 0 — Measure the gap **(1–2 weeks)**

1. Rebase the alternative-equations commits onto the current `olf-local-patches`.
2. Lift the `NEWTON_RAPHSON`-only gate behind a test-only switch so the GPU solver can run on an alternative-equations system.
3. Write a pattern comparison tool: build the CPU union pattern and the GPU superset for the same network, map both to variable-row numbering, report the entries in one and not the other, by cause.
4. Run it on the IEEE cases, pegase9241 and rte6515.
5. Census: how many buses and controls are ineligible on each network, by rule; how many contingencies fall back on the CPU side; how many go to the CPU on the GPU side.
6. Measure the cost of the stored zeros on the CPU: KLU fill and factorisation time with the GPU superset pattern against the legacy pattern.

Exit. A table of the pattern difference and of the coverage gap, and a go/no-go on CPU performance. Everything after depends on these numbers.

Phase 1 — Scatter as the storage **(2–3 weeks)**

1. Bring #1521 into the fork (or build on it once merged upstream).
2. Add the composed map "term derivative → Jacobian value index".
3. Under alternative equations, stop filtering inactive terms out of the maps: keep them, and let the state arrays decide the value. The maps are then never invalidated.
4. Rewrite the partial Jacobian update on top of the maps (re-scatter the entries of the touched slots).

Exit. Values and derivatives bit-identical to the current implementation on the OLF test suite; the PR's own benchmark not slower; no map rebuild during a security analysis on an eligible network.

Phase 2 — Variable-owned slots in OLF (6–10 weeks)

One sub-step per family, each removing one exclusion, each validated against the legacy modelling on the whole OLF test suite:

1. Introduce the slot abstraction; move the existing alternatives (reactive balance, voltage target, `DISTR_Q`, disabled bus) onto it. No behaviour change.
2. Slack and reference bus as alternatives of the φ slot; slack relocation becomes a value change.
3. Generators with slope; merged and hidden voltage controls.
4. Transformer voltage control and shunt voltage control, on the tap and susceptance slots.
5. Phase control on the phase-shift slot.
6. Zero-impedance branches: dummy slots, spanning-tree re-election as an alternative switch. This is the hardest step; the earlier attempt on the fork was reverted.
7. Converters and fictitious buses, or a documented decision to leave them out.

Exit. On rte6515 with the production parameters, a security analysis performs one structure build and no non-square fallback; results equal the legacy modelling within the solver tolerance.

Phase 3 — The contract (2–3 weeks)

1. Define `FixedStructureView` in OLF with the arrays of the previous section.
2. Make `JacobianMatrix` fill from it, so the CPU solver is its first consumer.
3. Property test: the matrix assembled from the view equals the matrix assembled by the term walk, for random states and random alternative selections.

Exit. The CPU load flow and security analysis run on the view only.

Phase 4 — The GPU reads the contract (4–6 weeks)

1. Import path: build `SharedStructure` from the view's pattern and maps.
2. Run both paths side by side and assert equality of pattern, destinations, row modes and targets on every test network. This is the gate.
3. Switch the default; keep the old extractor behind a switch for one release.
4. Delete `StableRowIndexing`, `ExtractionColumns`, the activation extractor, the relocation tables and the native pattern collection.

Exit. The GPU test suite passes with the old extraction removed; `GpuAcValidation` shrinks to a version check on the contract.

Phase 5 — Contingencies as data (6–8 weeks)

1. Compute the delta of a contingency in OLF without mutating the network.
2. CPU security analysis applies and undoes deltas instead of applying and restoring network state; this subsumes the "restore only impacted elements" work.
3. The GPU planner packs deltas into lanes; the mirrors in `batch/plan` are removed family by family (branch outage first, then injections, then voltage control re-merge).

Exit. The reject reasons that exist only because the mirror cannot express a case are gone; remaining rejects are genuine device limits.

Phase 6 — Upstream

Propose in this order, each standing on its own: the scatter extensions (after #1521), the slot abstraction with the existing alternatives, then one PR per family of phase 2, then the view. The GPU repository keeps working against the fork until each lands.

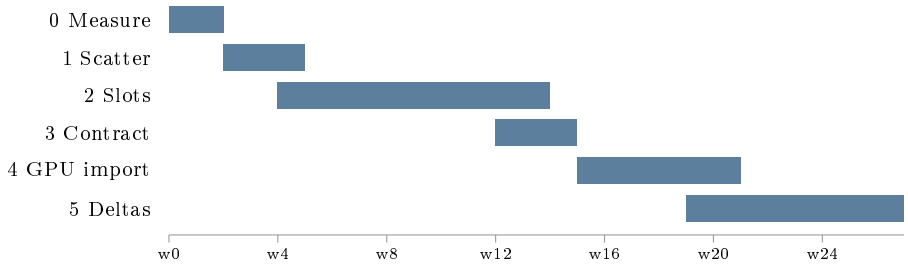


Figure 7: Indicative sequencing. Phases 1 and 2 overlap; phase 5 can start once the GPU reads the contract.

8 Risks and open questions

CPU cost of the union pattern. Stored zeros increase fill. The GPU already pays it; whether KLU gains more from one symbolic factorisation than it loses to fill is unmeasured (phase 0). `DISTR_Q` on large controller groups and the union of carrier candidates are the entries to watch.

Upstream acceptance. Without it, the maintenance burden moves from this repository to the fork rather than disappearing. The alternative-equations change is already about 3 700 lines in OLF's core `equations` package. The argument is the CPU gain; it has to be demonstrated first.

Zero-impedance spanning tree. OLF's tree depends on the history of contingencies; the GPU replays it. A stateless rule would be simpler for both sides but changes OLF behaviour.

Relocations that exist for numerics. The cross identity and the zero-diagonal repair serve factorisation without pivoting. They may belong on the GPU side as derived data rather than in the shared model.

Relation to #1521. It is a draft with no description; its direction may change. Phase 1 should be discussed with its author before building on it.

Incremental LU. The branch's notes say incremental LU updates on alternative switches were benchmarked and did not pay. Not revisited here.

Scope of a first upstream proposal. The smallest useful unit is probably the slot abstraction with today's alternatives and the partial update, on eligible networks only.

References

- OLF fork: branches `complementary-equations`, `alternative-equations-and-sa-mt-copy`, `experiment/alt-eq-contingency-fallback`.
- Upstream draft PR #1521, "Equation array, scatter approach": https://github.com/powsybl/powsybl-open-loadflow/pull/1521.
- This repository: `docs/gpu-vs-olf.md` (§1 one fixed matrix, §2 stable rows, row state, placement, factorisation without pivoting); `docs/hybrid-outer-loop-production-design.md` §11; `native/cuda/structure/shared_structure.cu`.